

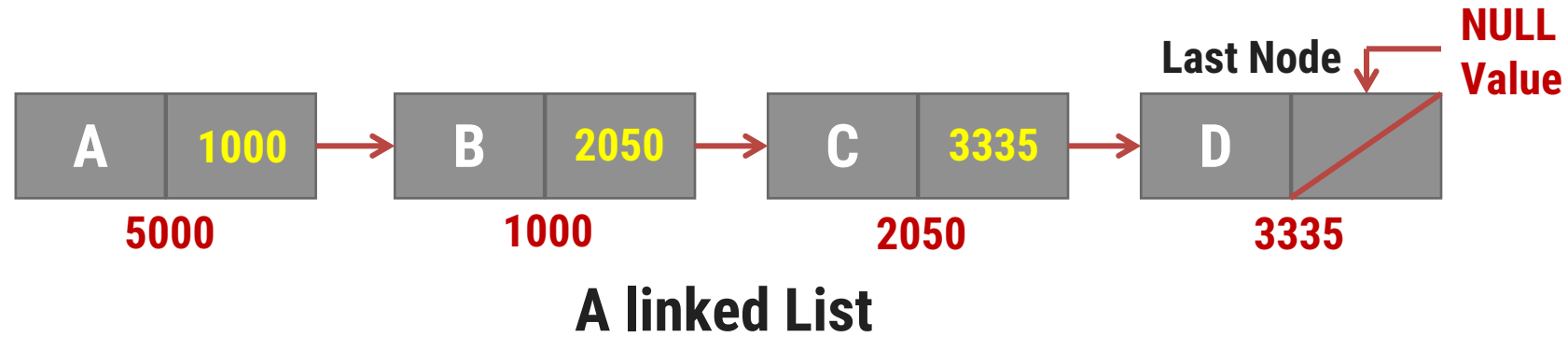
Linear Data Structure (Linked List)



Linked Storage Representation

- ▶ There are many applications where **sequential allocation** method is **unacceptable** because of following characteristics
 - ↳ **Unpredictable storage** requirement
 - ↳ **Extensive manipulation** of stored data
- ▶ One method of obtaining the address of node is to store address in computer's main memory, we refer this addressing mode as **pointer of link addressing**.
- ▶ A simple way to represent a linear list is to expand each node to contain a link or pointer to the next node. This representation is called one-way chain or Singly Linked Linear List.

Linked Storage Representation

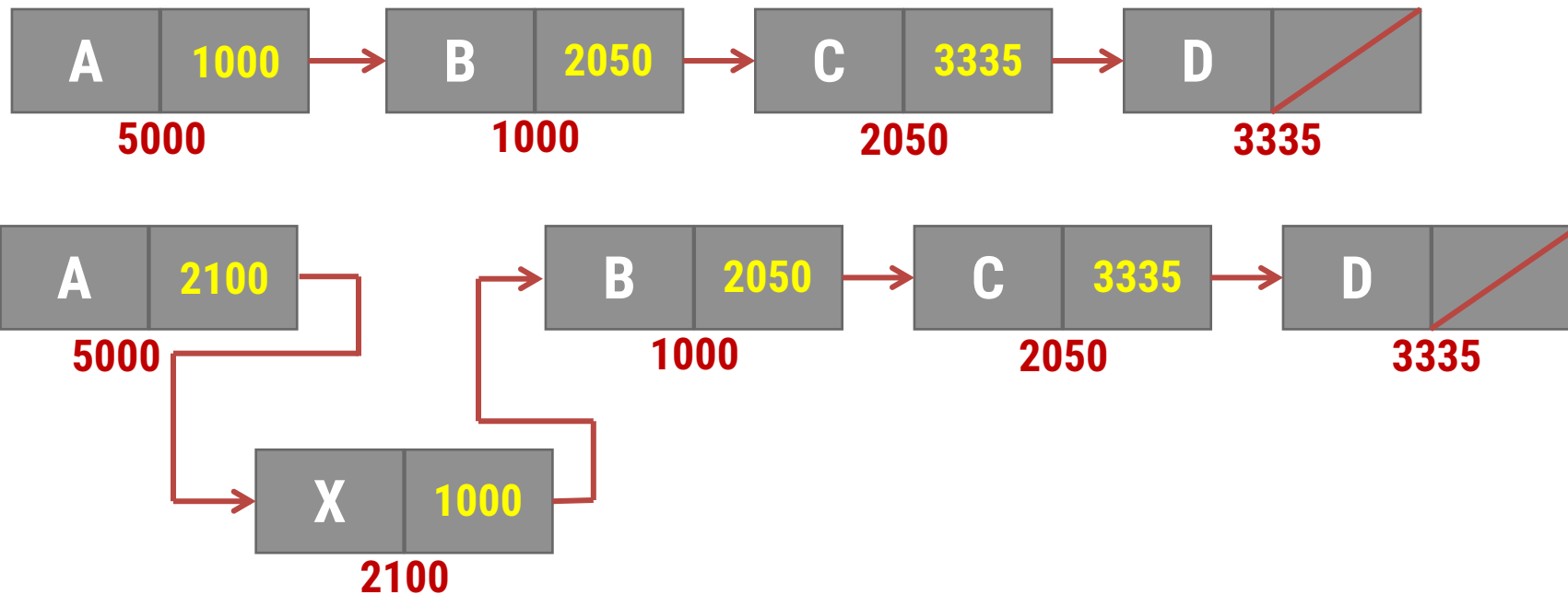


- ▶ The linked allocation method of storage can result in both efficient use of computer storage and computer time.
 - ➔ A linked list is a **non-sequential collection** of data items.
 - ➔ Each **node** is **divided** into **two parts**, the **first part** represents the **information** of the element and the **second part** contains the **address of the next node**.
 - ➔ The **last node** of the list does not have successor node, so **null value** is stored as the address.
 - ➔ It is possible for a list to have no nodes at all, such a list is called empty list.

Pros & Cons of Linked Allocation

► Insertion Operation

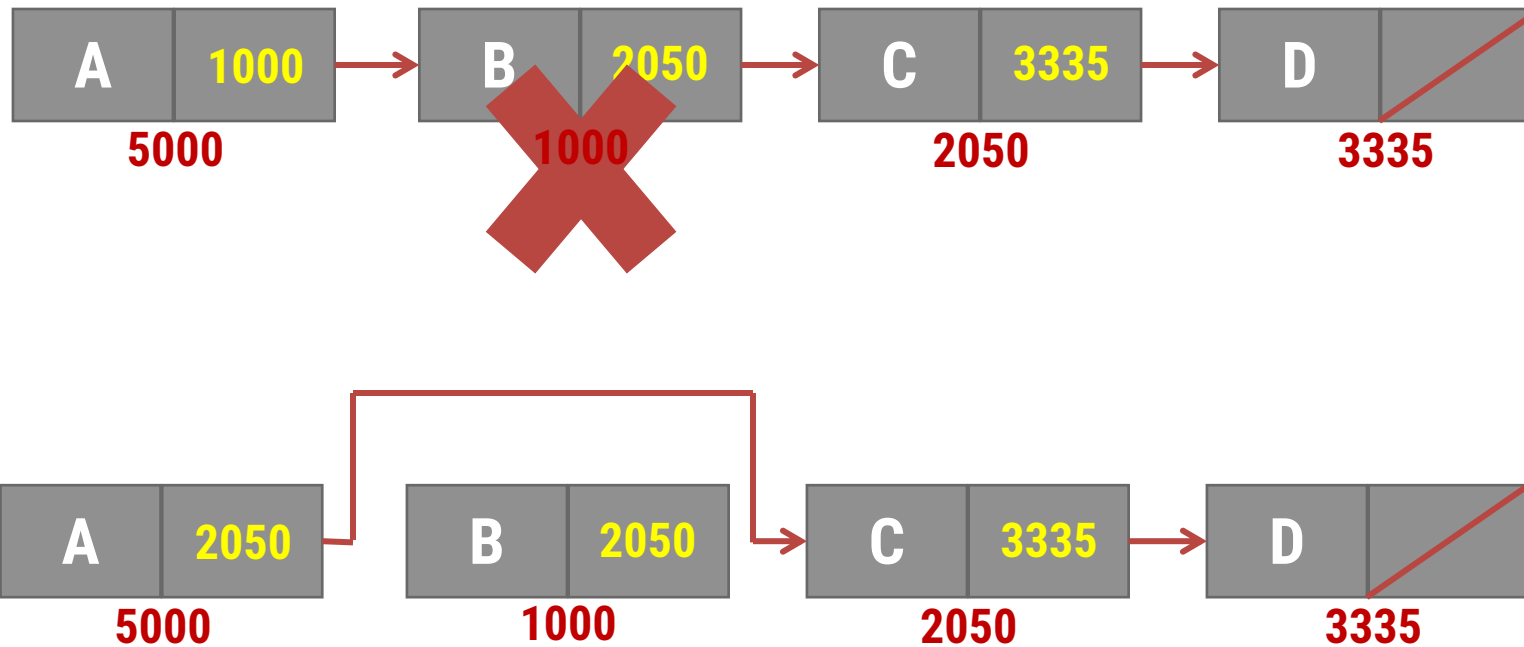
- We have an n elements in list and it is required to insert a new element between the first and second element, what to do with sequential allocation & linked allocation?
- Insertion operation is more efficient in Linked allocation.



Pros & Cons of Linked Allocation

► Deletion Operation

→ Deletion operation is more efficient in Linked Allocation



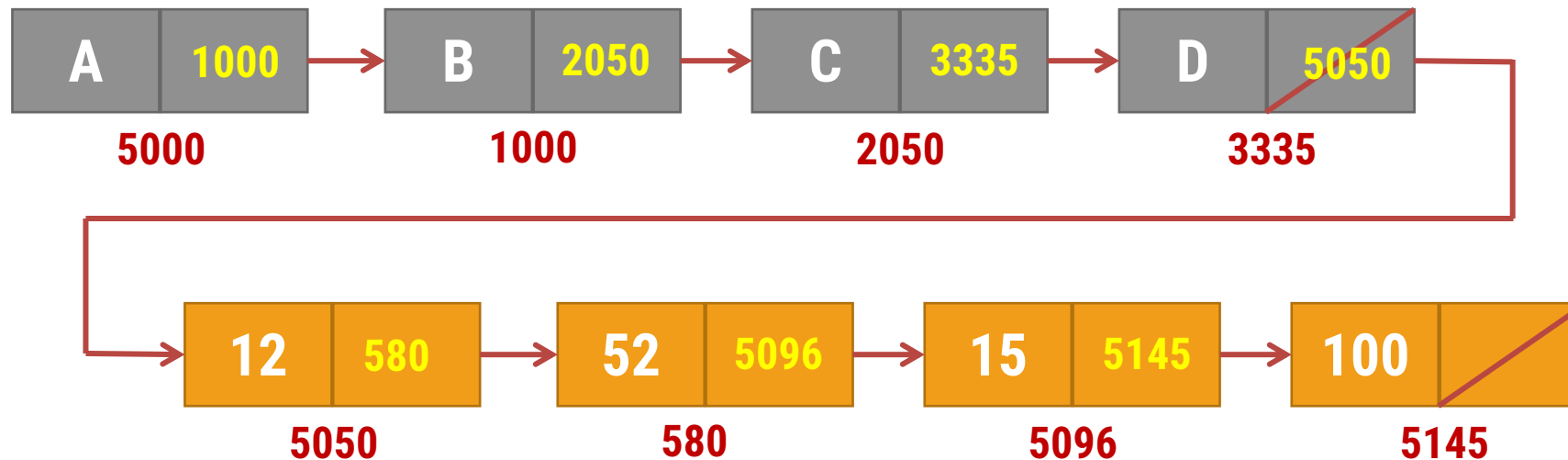
Pros & Cons of Linked Allocation

► Search Operation

- If **particular node in the list is required**, it is necessary to follow links from the first node onwards until the desired node is found, in this situation **it is more time consuming** to go through linked list than a sequential list.
- Search operation is more time consuming in Linked Allocation.

► Join Operation

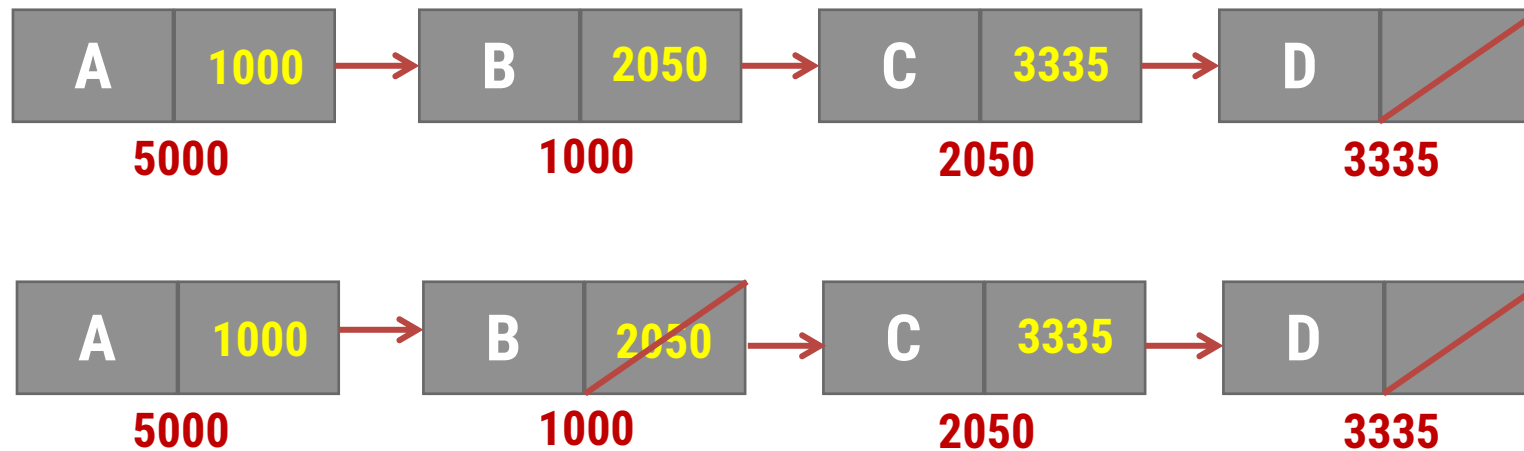
- Join operation is more efficient in Linked Allocation.



Pros & Cons of Linked Allocation

► Split Operation

→ Split operation is more efficient in Linked Allocation



- Linked list require **more memory** compared to array because along with value it stores pointer to next node.
- Linked lists are among the simplest and most common data structures. They can be used to implement other data structures like stacks, queues, and symbolic expressions, etc...

Operations & Type of Linked List

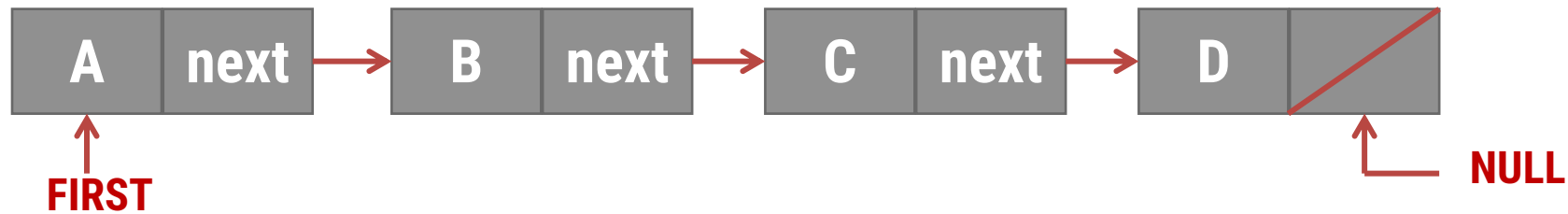
Operations on Linked List

- Insert
 - Insert at first position
 - Insert at last position
 - Insert into ordered list
- Delete
- Traverse list (Print list)
- Copy linked list

Types of Linked List

- Singly Linked List
- Circular Linked List
- Doubly Linked List

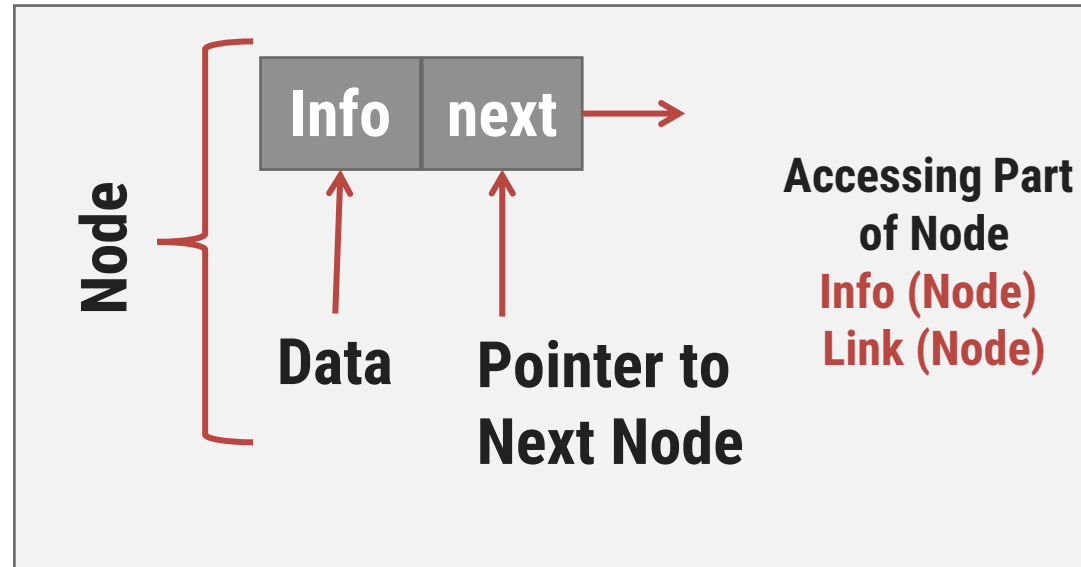
Singly Linked List



- ▶ It is basic type of linked list.
- ▶ Each node contains data and pointer to next node.
- ▶ Last node's pointer is null.
- ▶ First node address is available with pointer variable **FIRST(or START)**.
- ▶ **Limitation** of singly linked list is **we can traverse only in one direction**, forward direction.

Node Structure of Singly List

Typical Node



C Structure to represent a node

```
struct node
{
    int info;
    struct node *next;
};
```

Algorithms for singly linked list

1. Insert at first position
2. Insert at last position
3. Insert in Ordered Linked list
4. Delete Element
5. Copy Linked List

Malloc function

- ▶ An array is a collection of items stored at contiguous memory locations.

40	55	63	17	22	68	89	97	89
0	1	2	3	4	5	6	7	8

<- Array Indices

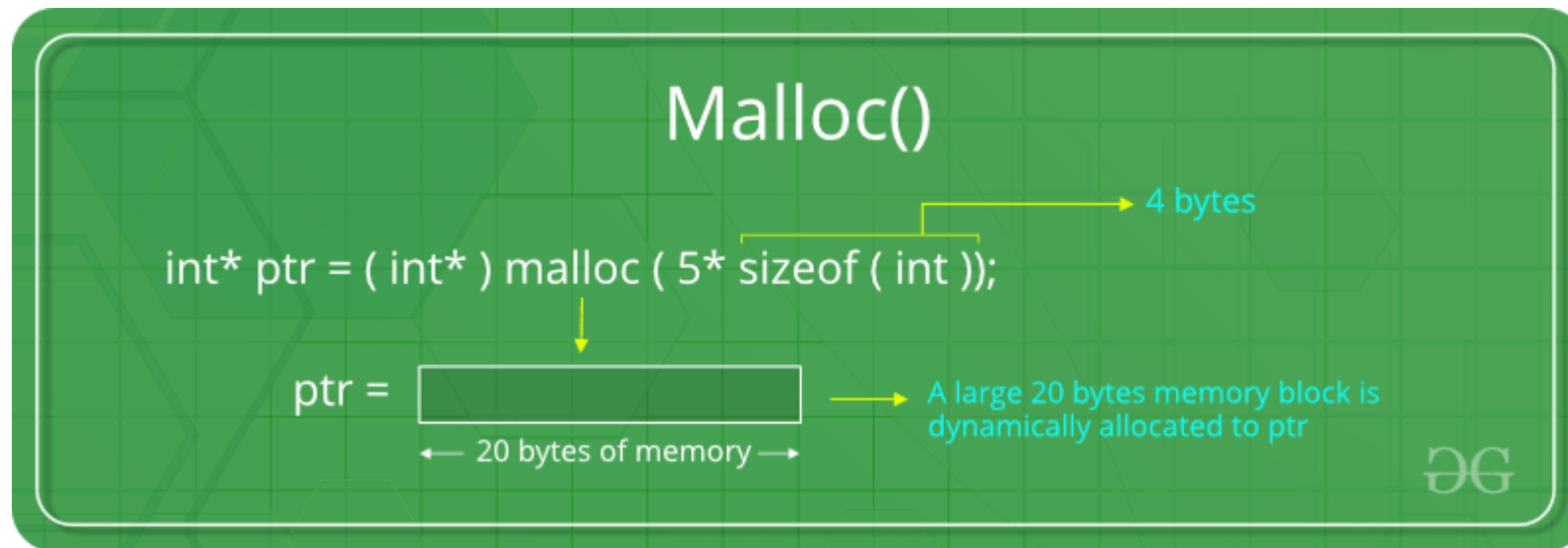
Array Length = 9
First Index = 0
Last Index = 8

- ▶ As can be seen, the length (size) of the array above is 9. But what if there is a requirement to change this length (size)? For example,
 - If there is a situation where only 5 elements are needed to be entered in this array. In this case, the remaining 4 indices are just wasting memory in this array. So there is a requirement to lessen the length (size) of the array from 9 to 5.
 - Take another situation. In this, there is an array of 9 elements with all 9 indices filled. But there is a need to enter 3 more elements in this array. In this case, 3 indices more are required. So the length (size) of the array needs to be changed from 9 to 12

Malloc function

- ▶ The “malloc” or “memory allocation” method in C is used to dynamically allocate a single large block of memory with the specified size. It returns a pointer of type void which can be cast into a pointer of any form.

```
ptr = (cast-type*) malloc(byte-size)
```



- ▶ In our case we want to create new node which is having structure(data+next pointer)

```
ptr = (node*) malloc(sizeof(node))
```

Function: INSERT(X, First)

- ▶ This function **inserts a new node at the first position** of Singly linked list.
- ▶ This function returns address of **FIRST** node.
- ▶ **X** is a new element to be inserted.
- ▶ **FIRST** is a **pointer to the first element** of a Singly linked linear list.
- ▶ Typical node contains **INFO** and **NEXT** fields.
- ▶ **NEW** is a temporary pointer variable.

Function: INSERT(X,FIRST) Cont...

1. [Obtain address of new Node in NEW pointer]
NEW $\leftarrow$ (node*) malloc(sizeof(node))
2. [Initialize fields of new node and its link to the list]
NEW->INFO $\leftarrow$ X
NEW->NEXT $\leftarrow$ FIRST
3. [Return address of new node]
Return (NEW)

Example: INSERT(50, FIRST)



FIRST $\leftarrow$ INSERT (X, FIRST)

2. [Initialize fields of new node and its link to the list]

NEW- $\rightarrow$ INFO $\leftarrow$ X

NEW- $\rightarrow$ NEXT $\leftarrow$ FIRST

3. [Return address of new node]

Return (NEW)

Function: INSEND(X, FIRST)

- ▶ This function **inserts** a new node at the **last position** of linked list.
- ▶ This function returns address of **FIRST** node.
- ▶ **X** is a new element to be inserted.
- ▶ **FIRST** is a **pointer to the first element** of a Singly linked linear list.
- ▶ Typical node contains **INFO** and **NEXT** fields.
- ▶ **NEW** is a temporary pointer variable.

Function: INSEND(X, First) Cont...

1. [Obtain address of new Node in NEW pointer]

`NEW ← (node*) malloc(sizeof(node))`

2. [Initialize fields of new node and its link to the list]

`NEW->INFO ← X`

`NEW->NEXT ← NULL` (because it will become the last node)

3. [Search for end of list]

`TEMP ← FIRST` (use TEMP pointer to traverse last node starting from FIRST node address)

Repeat while `TEMP->NEXT ≠ NULL`

`TEMP ← TEMP->NEXT`

4. [Set link field of last node to NEW]

`TEMP->NEXT ← NEW`

Function: INSEND(50, FIRST)

1. [Obtain address of new Node in NEW pointer]

$NEW \leftarrow (\text{node}^*) \text{ malloc}(\text{sizeof}(\text{node}))$

2. [Initialize fields of new node and its link to the list]

$NEW \rightarrow \text{INFO} \leftarrow X$

$NEW \rightarrow \text{NEXT} \leftarrow \text{NULL}$

3. [Search for end of list]

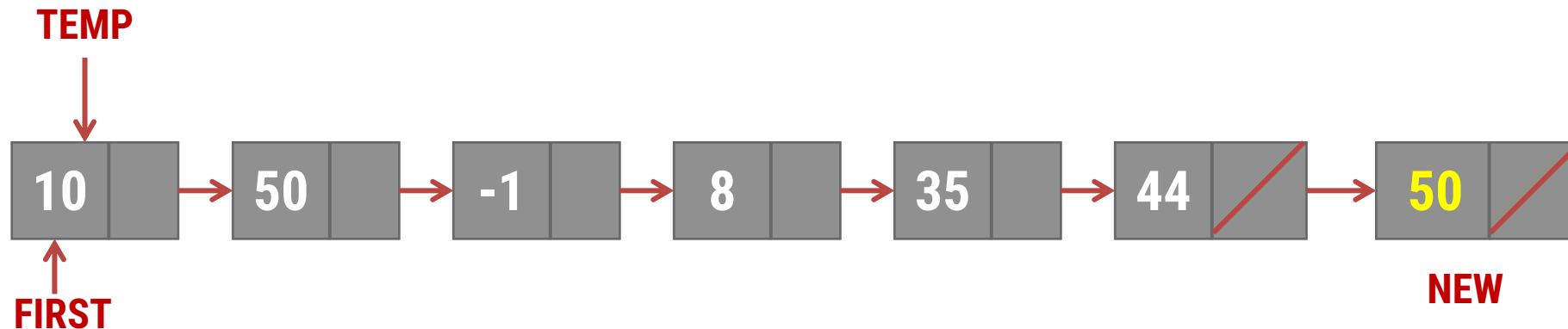
$\text{TEMP} \leftarrow \text{FIRST}$

Repeat while $\text{TEMP} \rightarrow \text{NEXT} \neq \text{NULL}$

$\text{TEMP} \leftarrow \text{TEMP} \rightarrow \text{NEXT}$

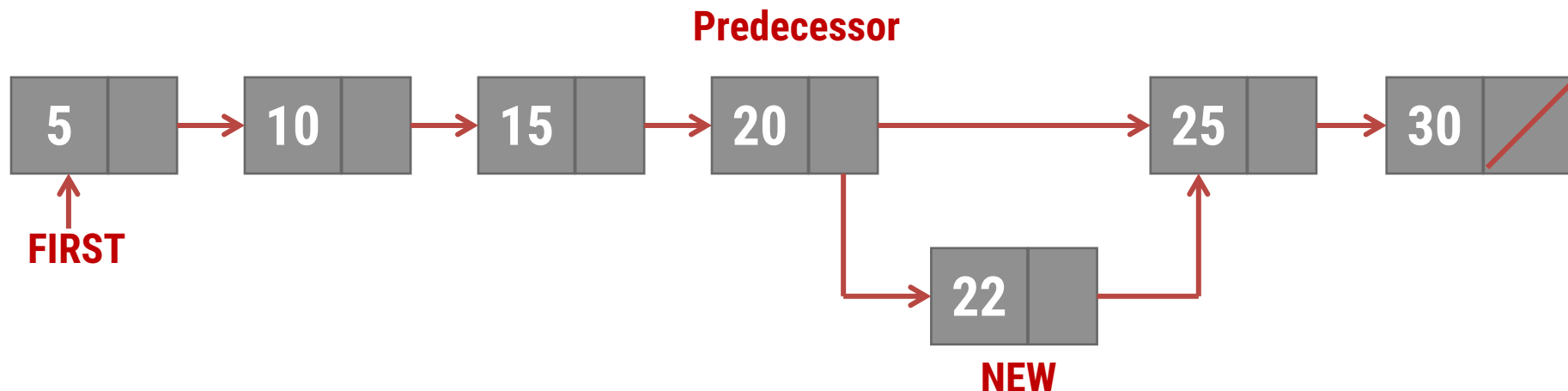
4. [Set link field of last node to NEW]

$\text{TEMP} \rightarrow \text{NEXT} \leftarrow \text{NEW}$



Function: INSORD(X, FIRST)

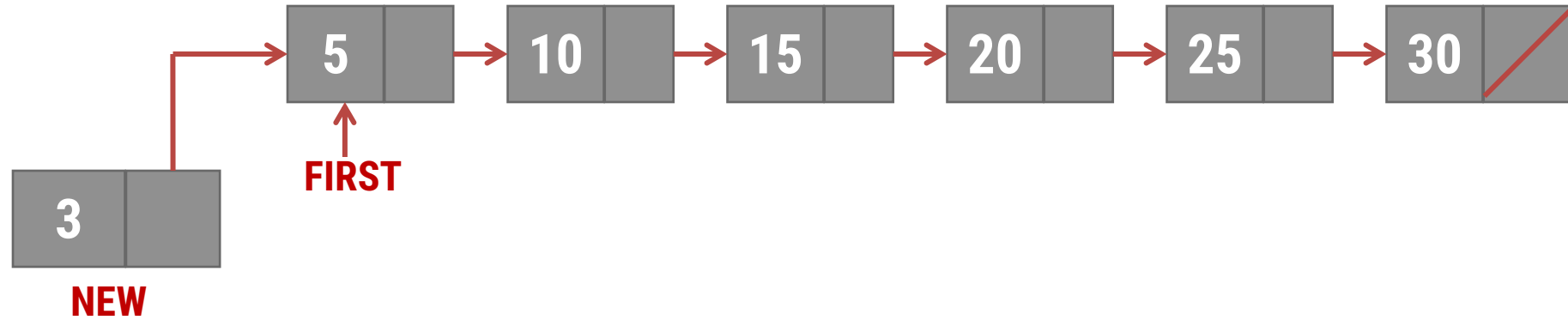
- ▶ This function **inserts** a new node such that linked list preserves the ordering of the terms in **increasing order** of their **INFO** field.
- ▶ This function returns address of **FIRST** node.
- ▶ **X** is a new element to be inserted.
- ▶ **FIRST** is a **pointer to the first element** of a Singly linked linear list.
- ▶ Typical node contains **INFO** and **NEXT** fields.
- ▶ **NEW** is a temporary pointer variable.



Function: INSORD(X, FIRST)

1. [Obtain address of new Node in NEW pointer]
NEW $\leftarrow$ (node*) malloc(sizeof(node))
2. [Initialize fields of new node and its link to the list]
NEW->INFO $\leftarrow$ X
NEW->NEXT $\leftarrow$ NULL (initially null)
3. [If new node info is smallest and needs to be added in the first position]
IF NEW->INFO $\leq$ FIRST->INFO
THEN NEW->NEXT $\leftarrow$ FIRST
Return (NEW)
4. [Initialize temporary pointer]
TEMP $\leftarrow$ FIRST
5. [Search for predecessor of new node]
Repeat while TEMP->NEXT $\neq$ NULL
& NEW->INFO $\geq$ (TEMP->NEXT)->INFO
TEMP $\leftarrow$ TEMP->NEXT
6. [Set link field of NEW node and its predecessor]
NEW->NEXT $\leftarrow$ TEMP->NEXT
TEMP->NEXT $\leftarrow$ NEW
7. [Return first node pointer]
Return (FIRST)

Function: INSORD(3, FIRST)



3. [If new node info is smallest and needs to be added in the first position]

IF NEW->INFO $\leq$ FIRST->INFO

THEN NEW->NEXT $\leftarrow$ FIRST

Return (NEW)

Function: INSORD(22, FIRST)

4. [Initialize temporary pointer]

$TEMP \leftarrow FIRST$

5. [Search for predecessor of new node]

Repeat while $TEMP \rightarrow NEXT \neq NULL$

& $NEW \rightarrow INFO \geq (TEMP \rightarrow NEXT) \rightarrow INFO$

$TEMP \leftarrow TEMP \rightarrow NEXT$

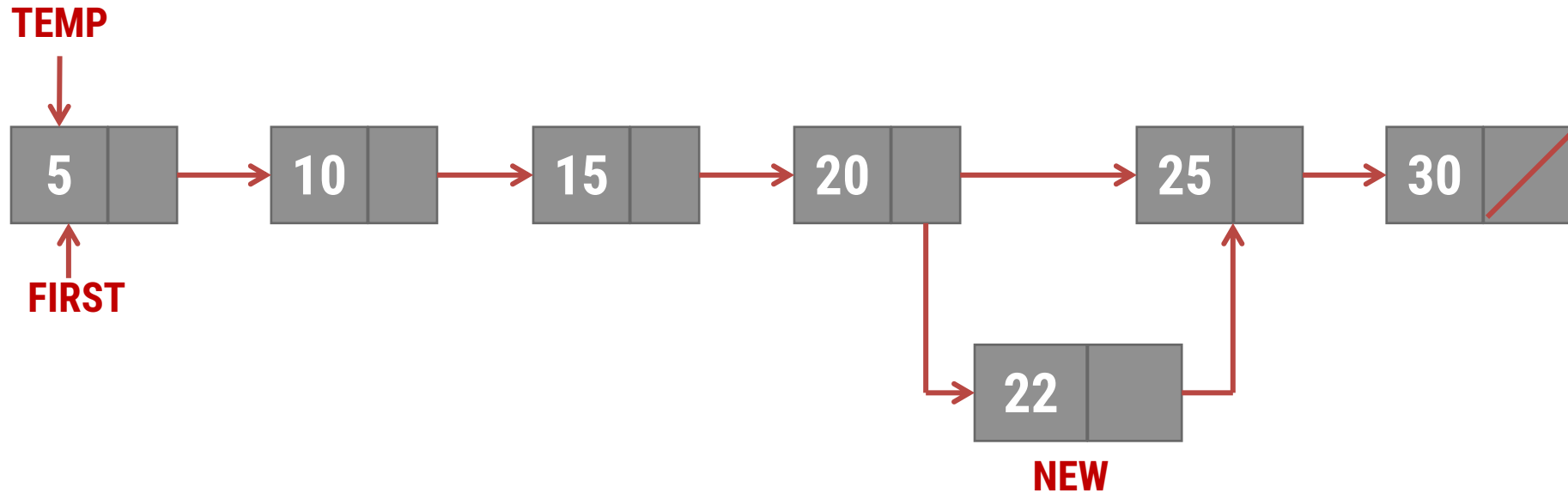
6. [Set link field of NEW node and its predecessor]

$NEW \rightarrow NEXT \leftarrow TEMP \rightarrow NEXT$

$TEMP \rightarrow NEXT \leftarrow NEW$

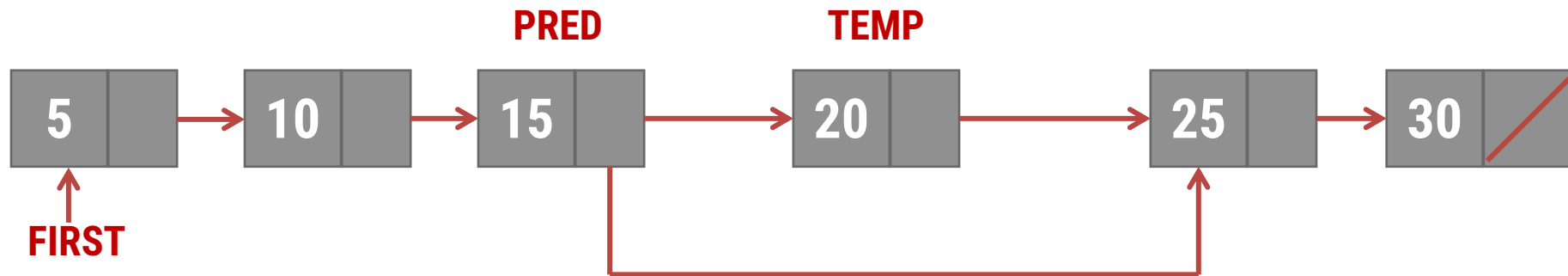
7. [Return first node pointer]

Return (FIRST)



Procedure: DELETE(X, FIRST)

- ▶ This algorithm **delete** a node whose address is given by variable **X**.
- ▶ **FIRST** is a **pointer to the first element** of a Singly linked linear list.
- ▶ Typical node contains **INFO** and **NEXT** fields.
- ▶ **TEMP** & **PRED** are temporary pointer variable.



Procedure: DELETE(X, FIRST)

1. [Is Empty list?]

```
IF      FIRST = NULL
THEN   write ('Underflow')
       Return
```

2. [Initialize search for X]

```
TEMP ← FIRST
```

3. [Find X]

```
Repeat thru step-5
while TEMP ≠ X and
TEMP->NEXT ≠ NULL
PRED ← TEMP
TEMP ← TEMP->NEXT
```

4. [End of the list?]

```
If      TEMP ≠ X
THEN   write ('Node not found')
       Return
```

5. [Delete X]

```
If    X = FIRST
THEN  FIRST ← FIRST->NEXT
ELSE  PRED->NEXT ← X->NEXT
```

6. [Free Deleted Node]

```
Free (X)
```

Procedure: DELETE(7541, FIRST)

2. [Initialize search for X]

TEMP $\leftarrow$ FIRST

3. [Find X]

Repeat thru step-5

while TEMP $\neq$ X and

TEMP- $\rightarrow$ NEXT $\neq$ NULL

PRED $\leftarrow$ TEMP

TEMP $\leftarrow$ TEMP- $\rightarrow$ NEXT

4. [End of the list?]

If TEMP $\neq$ X

THEN write ('Node not found')

Return

5. [Delete X]

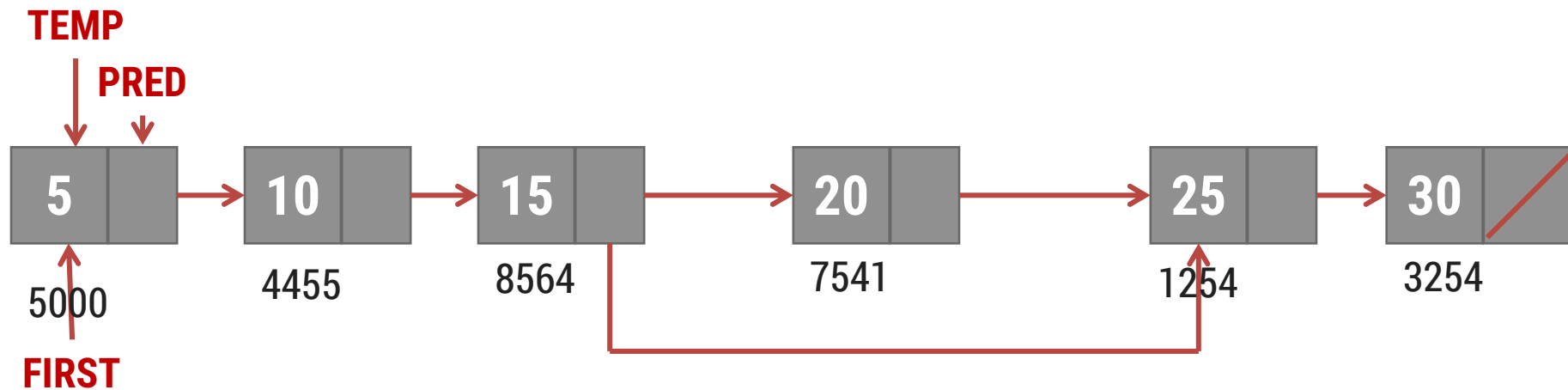
If X = FIRST

THEN FIRST $\leftarrow$ FIRST- $\rightarrow$ NEXT

ELSE PRED- $\rightarrow$ NEXT $\leftarrow$ X- $\rightarrow$ NEXT

6. [Free Deleted Node]

Free (X)



Function: COUNT_NODES(FIRST)

- ▶ This function **counts** number of nodes of the linked list and returns **COUNT**.
- ▶ **FIRST** is a **pointer to the first element** of a Singly linked linear list.
- ▶ Typical node contains **INFO** and **NEXT** fields.
- ▶ **TEMP** is a Temporary pointer variable.

1. [Is list Empty?]

```
IF      FIRST = NULL  
Then   COUNT ← 0  
       Return(COUNT)
```

2. [Initialize loop for a last node to update count]

```
TEMP ← FIRST
```

3. [Go for end of list]

```
Repeat while TEMP->NEXT ≠ NULL  
    TEMP ← TEMP->NEXT  
    COUNT ← COUNT + 1
```

4. [Return Count]

```
Return (COUNT)
```

Function: COPY (FIRST)

- ▶ This function **Copy** a Link List and creates new Linked List
- ▶ This function returns address of first node of newly created linked list.
- ▶ The **new list** is to contain **nodes** whose **information** and **pointer** fields are denoted by **FIELD** and **PTR**, respectively.
- ▶ The address of the **first node** in the newly created list is to be placed in **FIRST2**.(i.e. second linklist)
- ▶ **FIRST1** is a **pointer to the first element** of a Singly linked linear list.
- ▶ Typical node contains **INFO** and **NEXT** fields.
- ▶ **NEW, TEMP1** and **TEMP2** are temporary pointer variables.

Function: COPY (FIRST)

1. [Is Empty list?]

```
IF FIRST1 = NULL  
    Return(NULL)
```

ELSE

2. [Initialize Traversal and the first node is copied]

```
TEMP1 ← FIRST1  
NEW ← new node created  
NEW->INFO ← FIRST1->INFO  
FIRST2 ← NEW  
TEMP2 ← FIRST2
```

3. [Move to the next node and copy node]

```
While TEMP1->NEXT ≠ NULL  
    TEMP1 ← TEMP1->NEXT  
    NEW ← new node created  
    NEW->INFO ← TEMP1->INFO  
    TEMP2->NEXT ← NEW  
    TEMP2 ← TEMP2->NEXT
```

4. [Return second list]

```
RETURN(FIRST2)
```

Function: COPY (FIRST)

2. [Initialize Traversal and the first node is copied]

TEMP1 $\leftarrow$ FIRST1

NEW $\leftarrow$ new node created

NEW- $\rightarrow$ INFO $\leftarrow$ FIRST1- $\rightarrow$ INFO

FIRST2 $\leftarrow$ NEW

TEMP2 $\leftarrow$ FIRST2

3. [Move to the next node and copy node]

While TEMP1- $\rightarrow$ NEXT $\neq$ NULL

TEMP1 $\leftarrow$ TEMP1- $\rightarrow$ NEXT

NEW $\leftarrow$ new node created

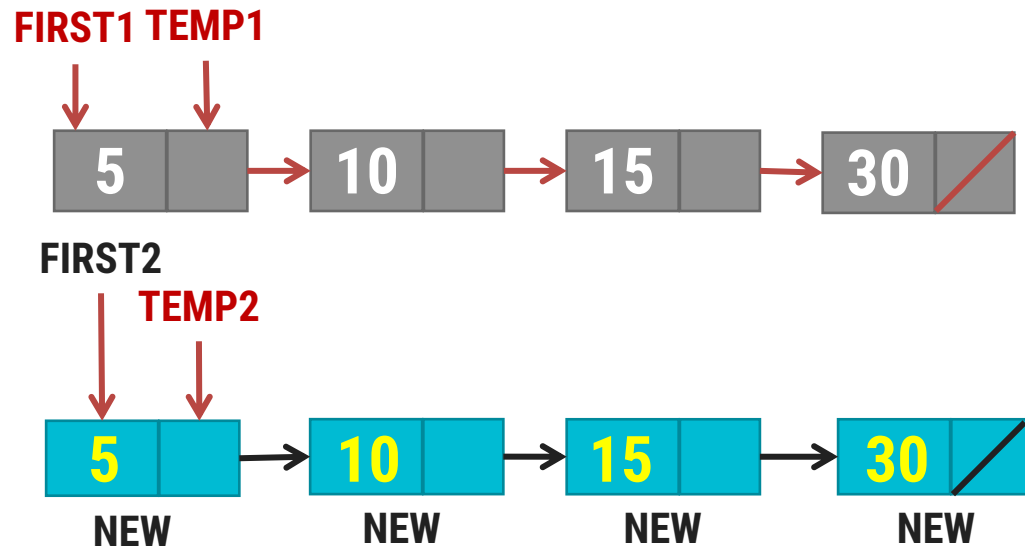
NEW- $\rightarrow$ INFO $\leftarrow$ TEMP1- $\rightarrow$ INFO

TEMP2- $\rightarrow$ NEXT $\leftarrow$ NEW

TEMP2 $\leftarrow$ TEMP2- $\rightarrow$ NEXT

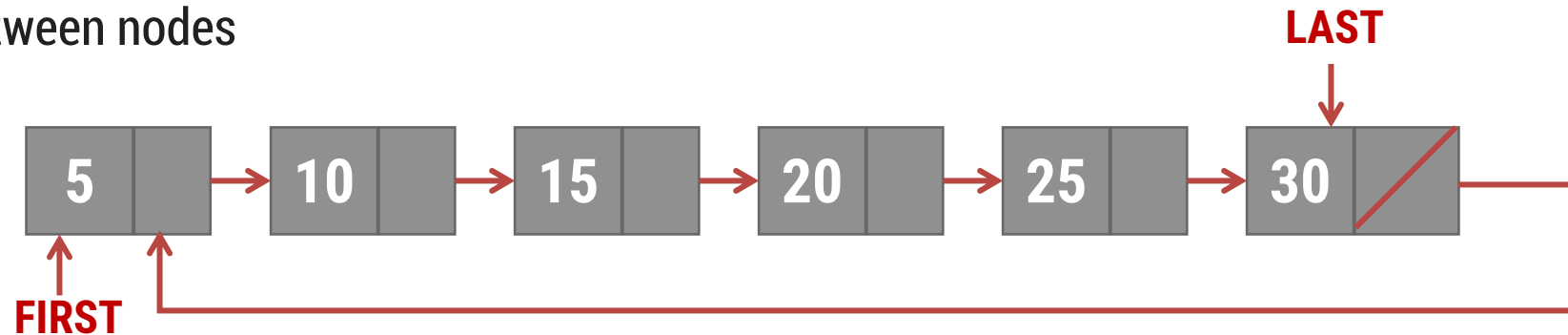
4. [Return second list]

RETURN(FIRST2)



Circularly Linked Linear List

- ▶ If we **replace NULL** pointer of the **last node** of Singly Linked Linear List with the **address of its first node**, that list becomes circularly linked linear list or **Circular List**.
- ▶ **FIRST** is the address of first node of Circular List
- ▶ **LAST** is the address of the last node of Circular List
- ▶ **Advantages of Circular List**
 - In circular list, every node is accessible from given node
 - It saves time when we have to go to the first node from the last node. It can be done in single step because there is no need to traverse the in between nodes. But in double linked list, we will have to go through in between nodes



Circularly Linked Linear List Cont...

► Disadvantages of Circular List

- ➞ It is not easy to reverse the linked list.
- ➞ If proper care is not taken, then the problem of infinite loop can occur.
- ➞ If we at a node and go back to the previous node, then we can not do it in single step. Instead we have to complete the entire circle by going through the in between nodes and then we will reach the required node.

► Operations on Circular List

- ➞ Insert at First
- ➞ Insert at Last
- ➞ Insert in Ordered List
- ➞ Delete a node

Procedure: CIR_INS_FIRST(X,FIRST,LAST)

- ▶ This procedure **inserts a new node at the first position** of Circular linked list.
- ▶ **X** is a new element to be inserted.
- ▶ **FIRST** and **LAST** are a **pointer to the first & last elements** of a Circular linked linear list, respectively.
- ▶ Typical node contains **INFO** and **NEXT** fields.
- ▶ **NEW** is a temporary pointer variable.

Procedure: CIR_INS_FIRST(X,FIRST,LAST)

1. [Creates a new empty node]

$NEW \leftarrow \text{NODE}$

2. [Initialize fields of new node and its link]

$NEW \rightarrow \text{INFO} \leftarrow X$

IF $\text{FIRST} = \text{NULL}$

THEN $NEW \rightarrow \text{NEXT} \leftarrow NEW$

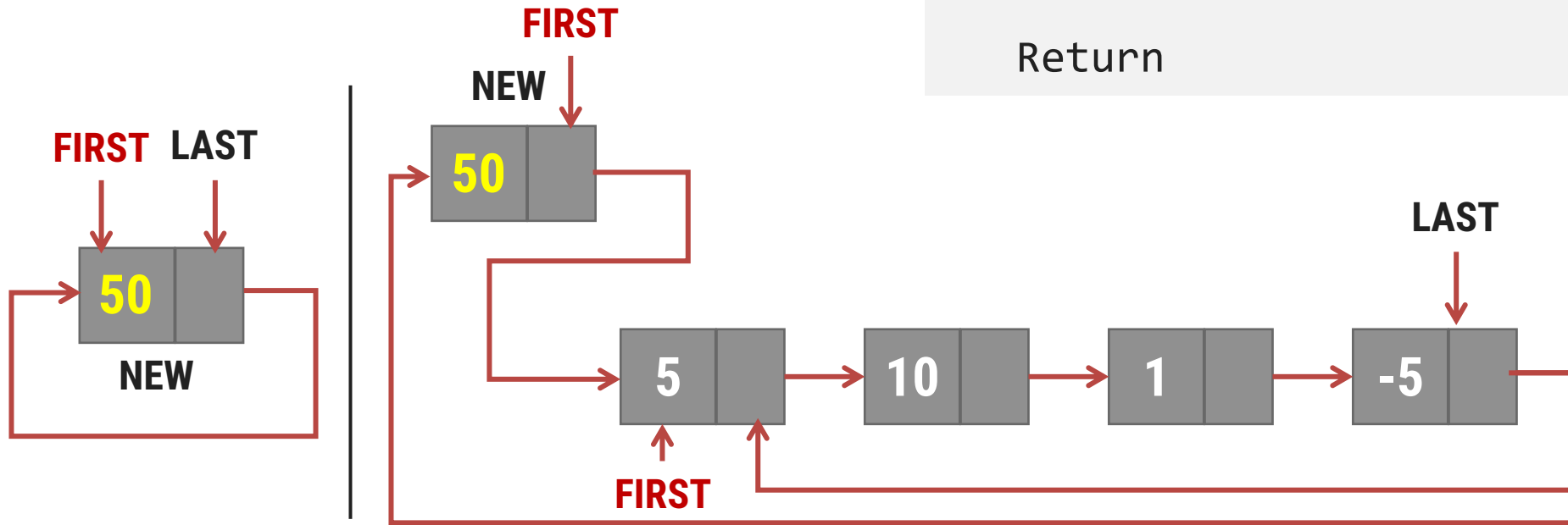
$\text{FIRST} \leftarrow \text{LAST} \leftarrow NEW$

ELSE $\text{LINK}(NEW) \leftarrow \text{FIRST}$

$\text{LINK}(\text{LAST}) \leftarrow NEW$

$\text{FIRST} \leftarrow NEW$

Return



Procedure: CIR_INS_LAST(X,FIRST,LAST)

- ▶ This procedure **inserts a new node at the last position** of Circular linked list.
- ▶ **X** is a new element to be inserted.
- ▶ **FIRST** and **LAST** are a **pointer to the first & last elements** of a Circular linked linear list, respectively.
- ▶ Typical node contains **INFO** and **LINK** fields.
- ▶ **NEW** is a temporary pointer variable.

Procedure: CIR_INS_LAST(X,FIRST,LAST)

1. [Creates a new empty node]

NEW $\leftarrow$ NODE

2. [Initialize fields of new node and its link]

NEW- $\rightarrow$ INFO $\leftarrow$ X

IF FIRST = NULL

THEN NEW- $\rightarrow$ NEXT $\leftarrow$ NEW

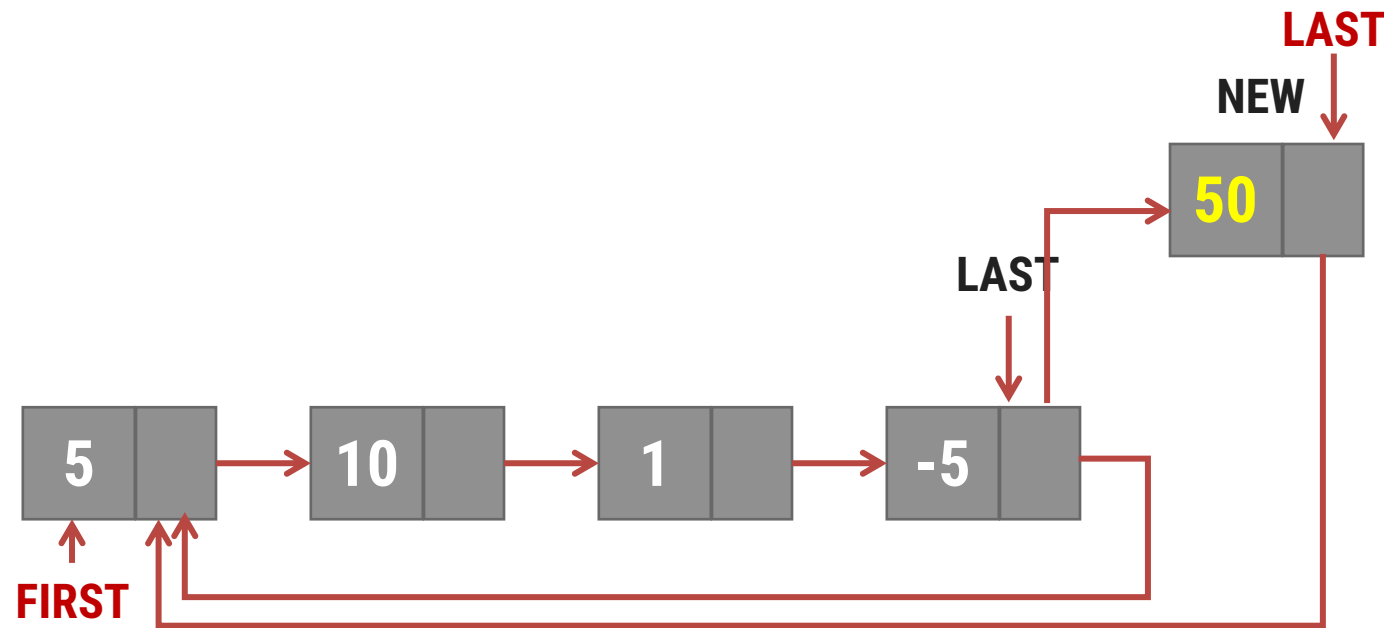
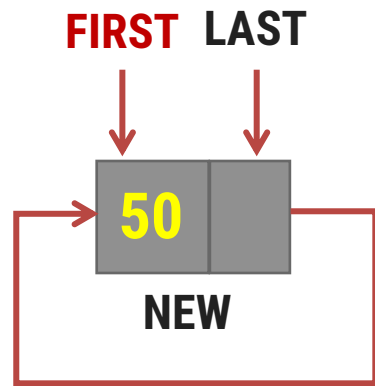
FIRST $\leftarrow$ LAST $\leftarrow$ NEW

ELSE NEW- $\rightarrow$ NEXT $\leftarrow$ FIRST

LAST- $\rightarrow$ NEXT $\leftarrow$ NEW

LAST $\leftarrow$ NEW

Return



Procedure: CIR_INS_ORD(X,FIRST,LAST)

- ▶ This function **inserts** a new node such that linked list preserves the ordering of the terms in **increasing order** of their **INFO** field.
- ▶ **X** is a new element to be inserted.
- ▶ **FIRST** and **LAST** are a **pointer to the first & last elements** of a Circular linked linear list, respectively.
- ▶ Typical node contains **INFO** and **NEXT** fields.
- ▶ **NEW** is a temporary pointer variable.

Procedure: CIR_INS_ORD(X,FIRST,LAST)

1. [Create New Empty Node]

NEW $\leftarrow$ NODE

2. [Copy information content into new node]

INFO(NEW) $\leftarrow$ X

3. [Is Linked List Empty?]

IF FIRST = NULL

THEN NEW- $\rightarrow$ NEXT $\leftarrow$ NEW

FIRST $\leftarrow$ LAST $\leftarrow$ NEW

Return

4. [Does new node data is less than the FIRST node]

IF NEW- $\rightarrow$ INFO $\leq$ FIRST- $\rightarrow$ INFO

THEN NEW - $\rightarrow$ NEXT $\leftarrow$ FIRST

LAST- $\rightarrow$ NEXT $\leftarrow$ NEW

FIRST $\leftarrow$ NEW

Return

5. [Initialize Temporary Pointer]

TEMP $\leftarrow$ FIRST

6. [Search for Predecessor of new node]

Repeat while TEMP $\neq$ LAST &

NEW- $\rightarrow$ INFO $\geq$ (TEMP- $\rightarrow$ NEXT)- $\rightarrow$ INFO

TEMP $\leftarrow$ TEMP- $\rightarrow$ NEXT

7. [Set link field of NEW node and its Predecessor]

NEW- $\rightarrow$ NEXT $\leftarrow$ TEMP- $\rightarrow$ NEXT

TEMP- $\rightarrow$ NEXT $\leftarrow$ NEW

IF TEMP = LAST

THEN LAST $\leftarrow$ NEW

8. [Finished]

Return

Procedure: CIR_INS_ORD(3,FIRST,LAST)

1. [Create New Empty Node]

NEW $\leftarrow$ NODE

2. [Copy information content into new node]

INFO(NEW) $\leftarrow$ X

3. [Is Linked List Empty?]

IF FIRST = NULL

THEN NEW- $\rightarrow$ NEXT $\leftarrow$ NEW

FIRST $\leftarrow$ LAST $\leftarrow$ NEW

Return

4. [Does new node data is less than the FIRST node]

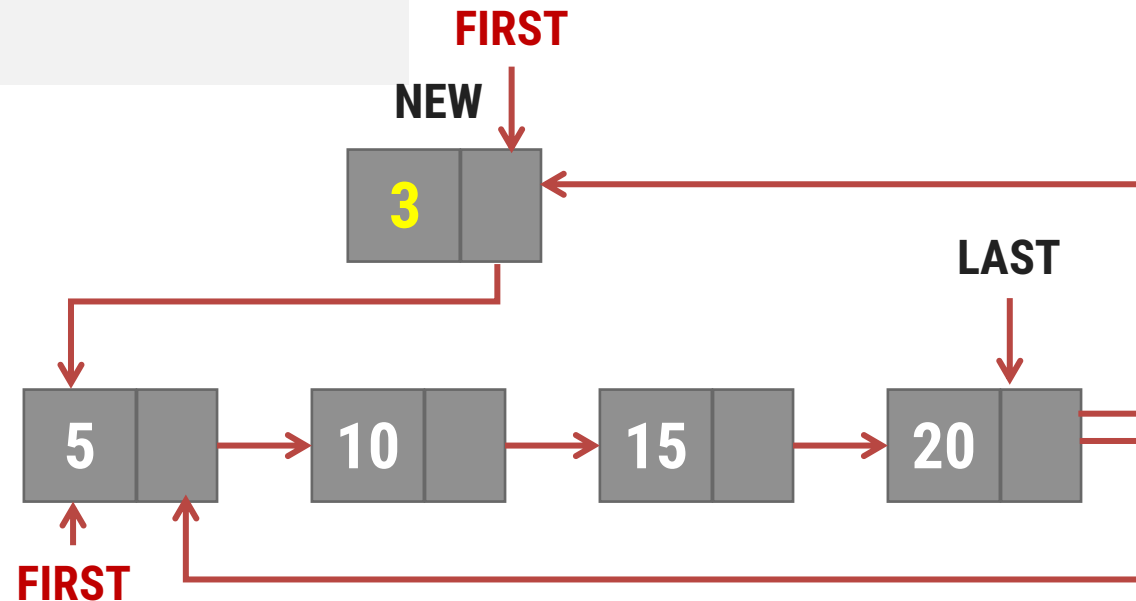
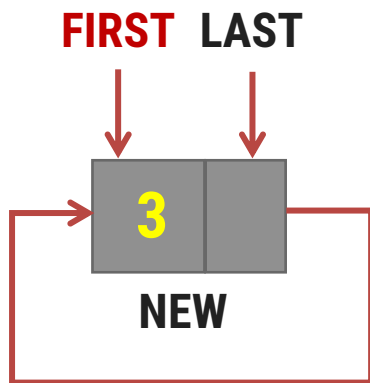
IF NEW- $\rightarrow$ INFO $\leq$ FIRST- $\rightarrow$ INFO

THEN NEW - $\rightarrow$ NEXT $\leftarrow$ FIRST

LAST- $\rightarrow$ NEXT $\leftarrow$ NEW

FIRST $\leftarrow$ NEW

Return



Procedure: CIR_INS_ORD(18,FIRST,LAST)

5. [Initialize Temporary Pointer]

TEMP $\leftarrow$ FIRST

6. [Search for Predecessor of new node]

Repeat while TEMP $\neq$ LAST &
NEW- $\rightarrow$ INFO $\geq$ (TEMP- $\rightarrow$ NEXT)- $\rightarrow$ INFO
TEMP $\leftarrow$ TEMP- $\rightarrow$ NEXT

7. [Set link field of NEW node and its Predecessor]

NEW- $\rightarrow$ NEXT $\leftarrow$ TEMP- $\rightarrow$ NEXT

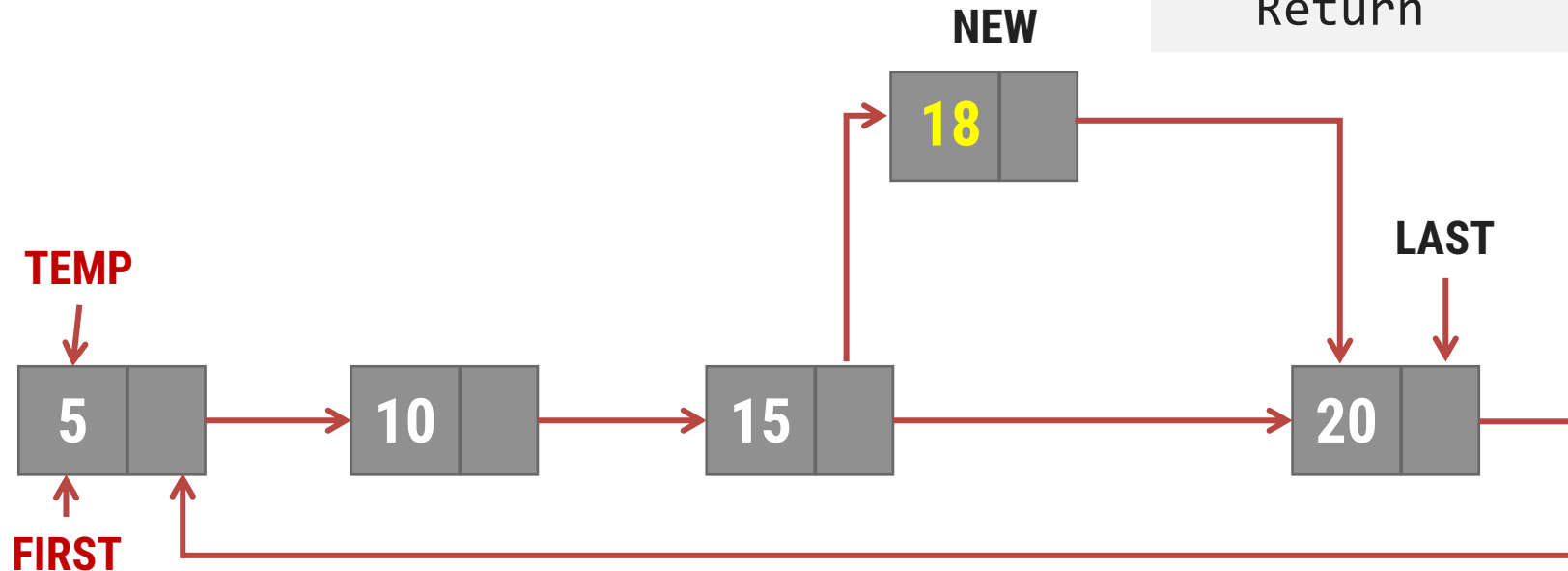
TEMP- $\rightarrow$ NEXT $\leftarrow$ NEW

IF TEMP = LAST

THEN LAST $\leftarrow$ NEW

8. [Finished]

Return



Procedure: CIR_DELETE(X,FIRST,LAST)

- ▶ This algorithm **delete** a node whose address is given by variable **X**.
- ▶ **FIRST** & **LAST** are **pointers to the First & Last elements** of a Circular linked list, respectively.
- ▶ Typical node contains **INFO** and **NEXT** fields.
- ▶ **TEMP** & **PRED** are temporary pointer variable.

Procedure: CIR_DELETE(X,FIRST,LAST)

1. [Is Empty List?]

```
IF FIRST = NULL
THEN write('Linked List is
Empty')
Return
```

2. [Initialize Search for X]

```
TEMP ← FIRST
```

3. [Find X]

```
Repeat thru step 5
while TEMP≠X & TEMP≠LAST
```

4. [Update predecessor marker]

```
PRED ← TEMP
```

5. [Move to next node]

```
TEMP ← TEMP->NEXT
```

6. [End of Linked List?]

```
IF TEMP ≠ X
THEN write('Node not found')
Return
```

7. [Delete X]

```
IF X = FIRST
THEN FIRST ← FIRST-TEMP
LAST-NEXT ← FIRST
ELSE PRED-NEXT ← X->NEXT
IF X = LAST
THEN LAST ← PRED
```

8. [Free Deleted Node]

```
Free (X)
```

Procedure: CIR_DELETE(7541,FIRST,LAST)

1. [Is Empty List?]

```
IF FIRST = NULL  
THEN write('Linked List is Empty')  
Return
```

2. [Initialize Search for X]

```
TEMP ← FIRST
```

3. [Find X]

```
Repeat thru step 5  
while TEMP ≠ X & TEMP ≠ LAST
```

4. [Update predecessor marker]

```
PRED ← TEMP
```

5. [Move to next node]

```
TEMP ← TEMP->NEXT
```

6. [End of Linked List?]

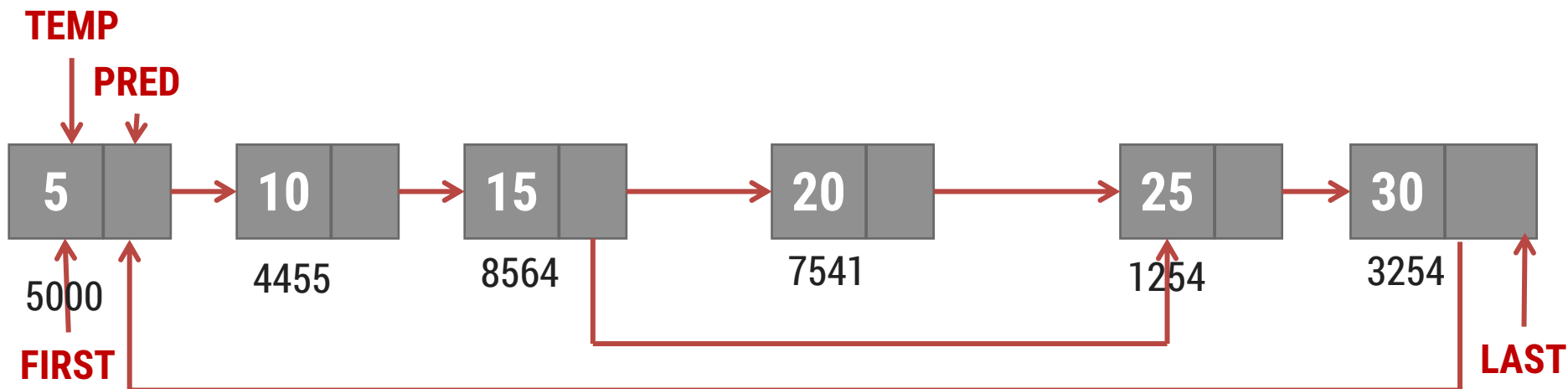
```
IF TEMP ≠ X  
THEN write('Node not found')  
Return
```

7. [Delete X]

```
IF X = FIRST  
THEN FIRST ← FIRST-TEMP  
LAST-NEXT ← FIRST  
ELSE PRED-NEXT ← X->NEXT  
IF X = LAST  
THEN LAST ← PRED
```

8. [Free Deleted Node]

```
Free (X)
```

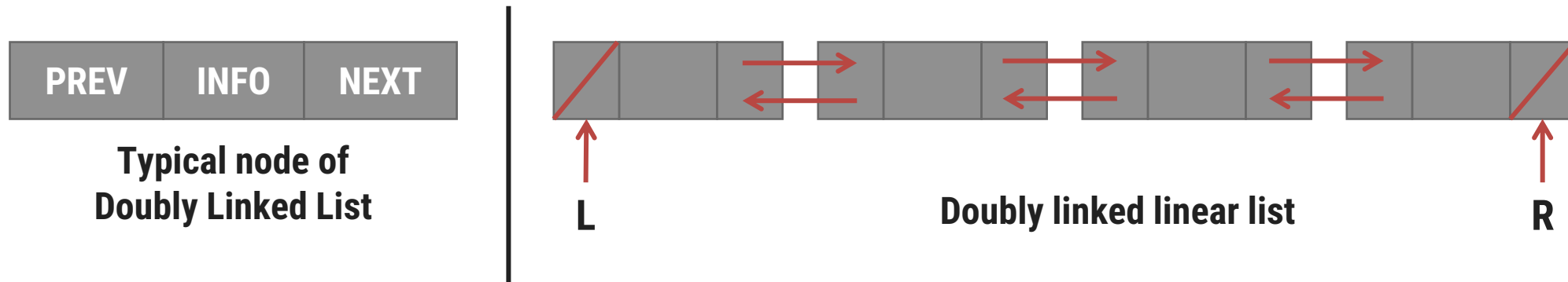


Doubly Linked Linear List

- ▶ In certain Applications, it is very desirable that a list be traversed in either forward or reverse direction.
- ▶ This property implies that each node must contain two link fields instead of usual one.
- ▶ The links are used to denote **Predecessor** and **Successor** of node.
- ▶ The link denoting its **predecessor** is called **prev**.
- ▶ The link denoting its **successor** is called **next**.
- ▶ A list containing this type of node is called **doubly linked list** or **two way chain**.

Doubly Linked Linear List

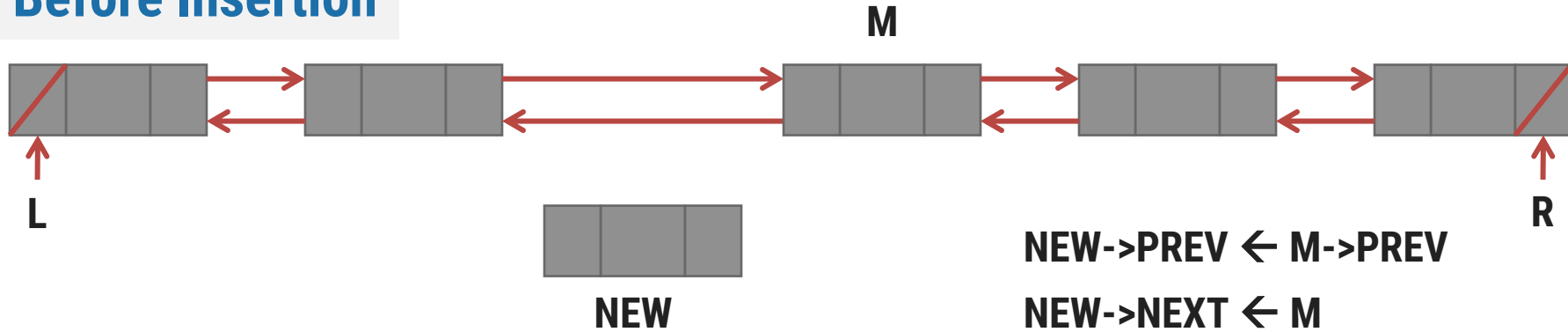
- ▶ Typical node of doubly linked linear list contains INFO, LPTR RPTR Fields
- ▶ **LPTR** is pointer variable pointing to Predecessor of a node
- ▶ **RPTR** is pointer variable pointing to Successor of a node
- ▶ Left most node of doubly linked linear list is called **L**, **PREV** of node **L is always NULL**
- ▶ Right most node of doubly linked linear list is called **R**, **NEXT** of node **R is always NULL**



Insert node in Doubly Linked List

Insertion in the middle of Doubly Linked Linear List

Before Insertion



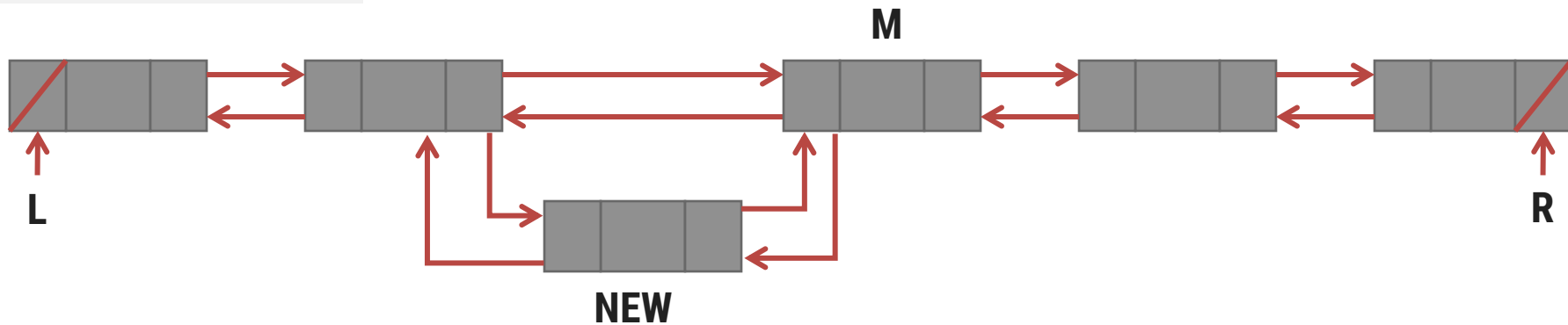
$NEW \rightarrow PREV \leftarrow M \rightarrow PREV$

$NEW \rightarrow NEXT \leftarrow M$

$M \rightarrow PREV \leftarrow NEW$

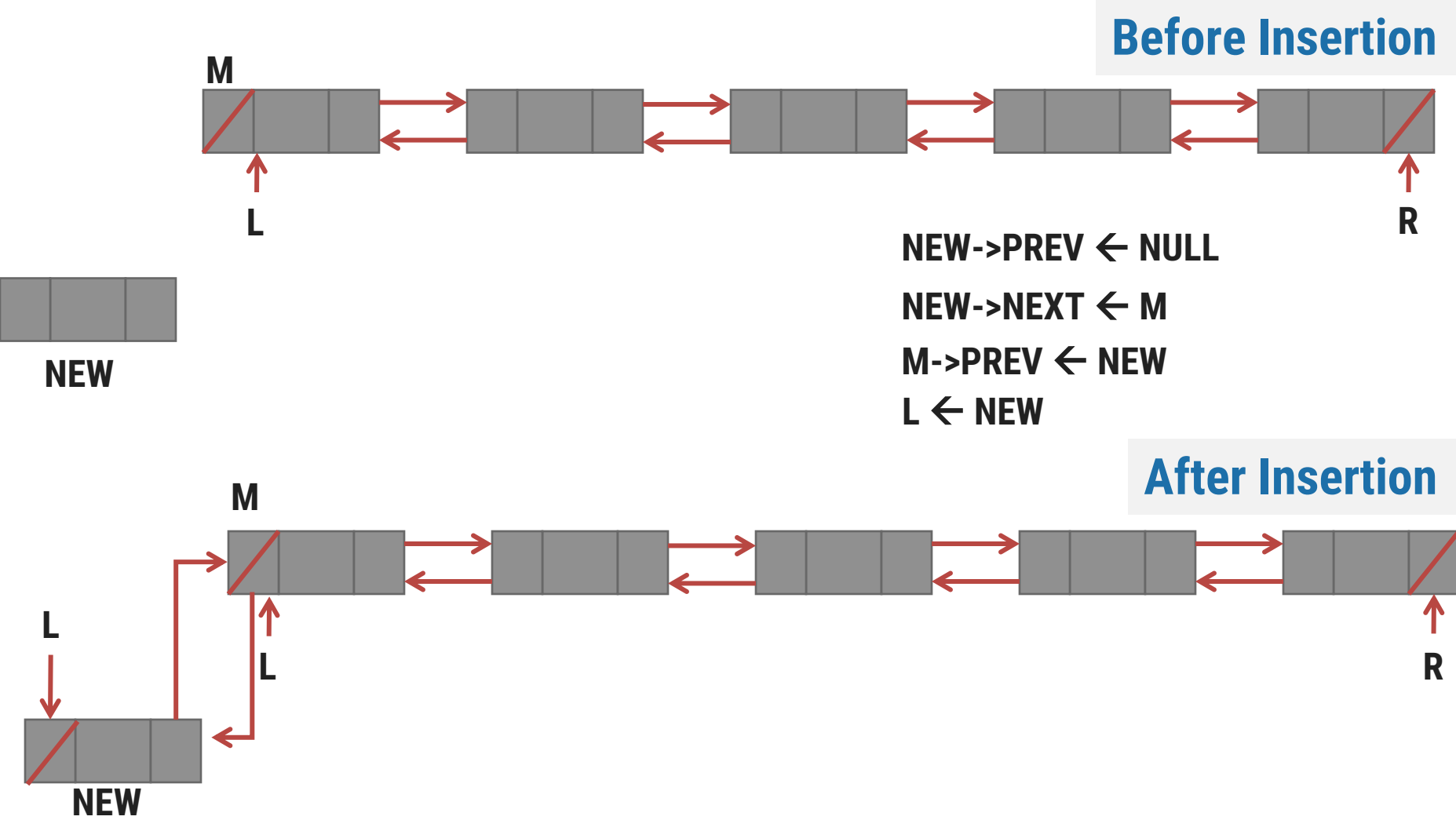
$(NEW \rightarrow PREV) \rightarrow NEXT \leftarrow NEW$

After Insertion



Insert node in Doubly Linked List

Left most insertion in Doubly Linked Linear List



Procedure: DOU_INS (L,R,M,X)

- ▶ This algorithm inserts a new node in doubly linked linear list.
- ▶ The **insertion** is to be **performed** to the **left of a specific node** with its **address** given by the pointer variable **M**.
- ▶ Typical node of doubly linked list contains following fields **PREV**, **NEXT** and **INFO**.
- ▶ **PREV** is pointer variable pointing to Predecessor of a node.
- ▶ **NEXT** is pointer variable pointing to Successor of a node.
- ▶ **L** & **R** are pointer variables pointing for Leftmost and Rightmost node of Linked List.
- ▶ **NEW** is the address of New Node.
- ▶ **X** is value to be inserted.

Procedure: DOU_INS (L,R,M,X)

1. [Create New Empty Node]

NEW $\leftarrow$ NODE

2. [Copy information field]

NEW- $\rightarrow$ INFO $\leftarrow$ X

3. [Insert into an empty list]

IF R = NULL (R is holding the address of the rightmost node if its null, it means list is empty)

THEN NEW- $\rightarrow$ PREV $\leftarrow$ NULL

NEW- $\rightarrow$ NEXT $\leftarrow$ NULL

L $\leftarrow$ R $\leftarrow$ NEW

Return

4. [Insertion at beginning]

IF M = L

THEN NEW- $\rightarrow$ PREV $\leftarrow$ NULL

NEW- $\rightarrow$ NEXT $\leftarrow$ M

M- $\rightarrow$ PREV $\leftarrow$ NEW

L $\leftarrow$ NEW

Return

5. [Insert in middle]

NEW- $\rightarrow$ PREV $\leftarrow$ LPTR(M)

NEW- $\rightarrow$ NEXT $\leftarrow$ M

M- $\rightarrow$ PREV $\leftarrow$ NEW

(NEW- $\rightarrow$ PREV)- $\rightarrow$ NEXT $\leftarrow$ NEW

Return

Procedure: DOU_INS (L,R,M,X)

1. [Create New Empty Node]

NEW $\leftarrow$ NODE

2. [Copy information field]

NEW- $\rightarrow$ INFO $\leftarrow$ X

3. [Insert into i^{th} position]

TEMP $\leftarrow$ L, $j \leftarrow 1$

while($j \neq i$)

TEMP $\leftarrow$ TEMP- $\rightarrow$ NEXT

$j \leftarrow j+1$

we reached to one node before the actual one

4. [Insert the node]

NEW- $\rightarrow$ PREV $\leftarrow$ TEMP

NEW- $\rightarrow$ NEXT $\leftarrow$ TEMP- $\rightarrow$ NEXT

TEMP- $\rightarrow$ NEXT $\leftarrow$ NEW

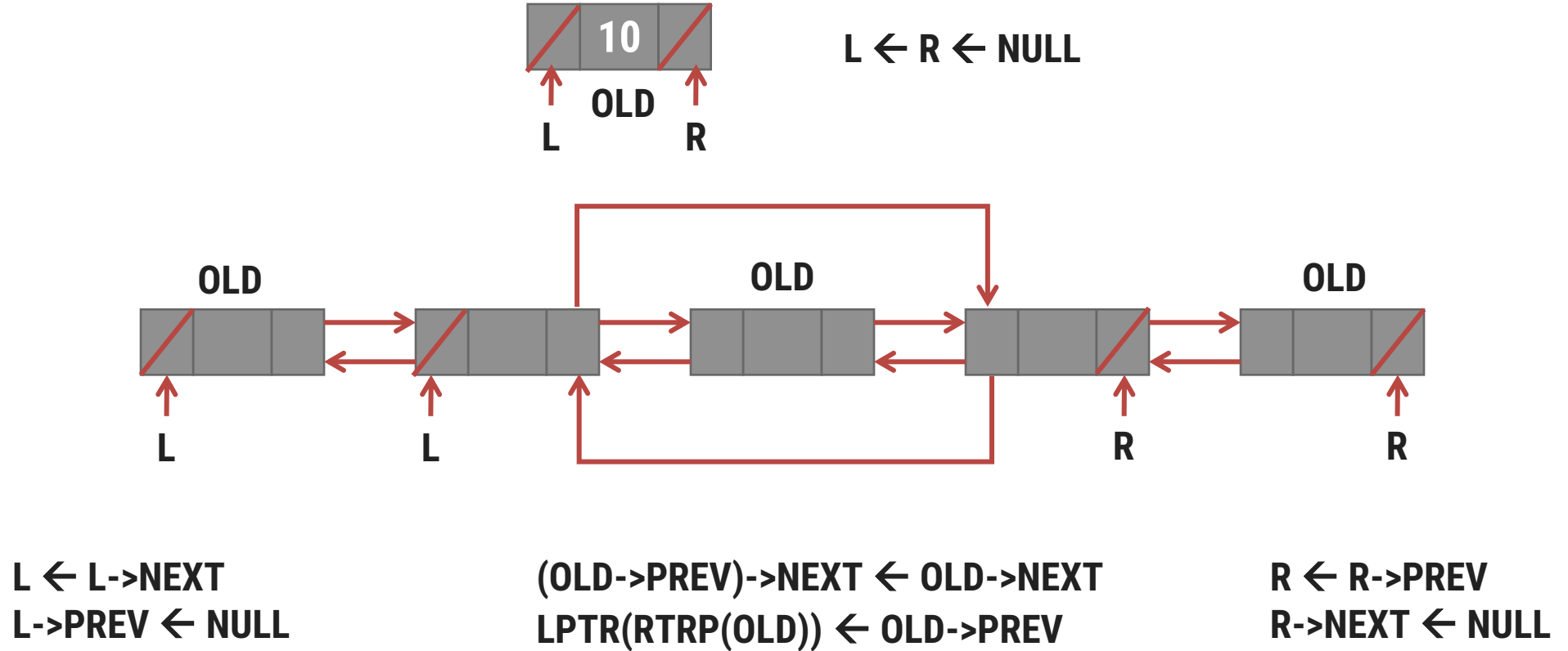
(NEW- $\rightarrow$ NEXT)- $\rightarrow$ PREV) $\leftarrow$ NEW

Return

PROCEDURE: DOU _DEL (L, R, OLD)

- ▶ This algorithm **deletes the node** whose **address** is contained in the variable **OLD**.
- ▶ Typical node of doubly linked list contains following fields **PREV**, **NEXT** and **INFO**.
- ▶ **PREV** is pointer variable pointing to Predecessor of a node.
- ▶ **NEXT** is pointer variable pointing to Successor of a node.
- ▶ **L** & **R** are pointer variables pointing for Leftmost and Rightmost node of Linked List.

Delete from Doubly Linked List



Procedure: DOU_DEL (L,R,M,X)

1. [Is underflow ?]

```
IF R=NULL  
THEN write ('UNDERFLOW')  
Return
```

2. [Delete node]

```
IF L = R (single node in list)  
THEN L ← R ← NULL
```

3. [Delete left most node]

```
IF OLD = L (left most node)  
THEN L ← L->NEXT  
L->PREV ← NULL
```

4. [Delete right most node]

```
IF OLD = R (right most)  
THEN R ← R->PREV  
R->NEXT ← NULL
```

5. [FREE deleted node ?]

```
FREE(OLD)
```

1. [delete i^{th} node]

```
TEMP ← L, j ← 1  
while(j ≠ i)  
TEMP ← TEMP->NEXT  
j ← j+1
```

we reached to one node before the actual one

4. [Insert the node]

```
OLD ← TEMP->NEXT  
TEMP->NEXT ← (OLD)->NEXT  
((OLD)->NEXT)->PREV ← TEMP  
FREE(OLD)
```

Time complexity analysis

Array:

- ▶ Accessing an element at a specific memory address takes $O(1)$ time, as its relative address can be calculated in constant time.
- ▶ Similarly editing any element of the array takes $O(1)$ time.
- ▶ Inserting a new element at a particular index in arrays is only possible when we skip all the elements before that index, which takes $O(N)$ time.
- ▶ Similarly, deletion operation also takes $O(N)$ time.
- ▶ Searching operation in arrays takes $O(N)$ time without any specific algorithm as we need to iterate and check every element in the array.

Time complexity analysis

Queue:

- ▶ To access or edit any element stored in a queue, the time taken is $O(N)$ as to reach any specific element, all the elements after it has to be removed.
- ▶ The searching operation also takes a total time of $O(N)$, as reaching any specific element isn't possible without popping the elements stored after it.
- ▶ Operations like insertion or deletion in a queue take constant time i.e., $O(1)$. Only the front element can be removed at a time and insertion can only take place at the back.

Time complexity analysis

Stack:

- ▶ To access or edit any element stored in a stack, the time taken is $O(N)$ as to reach any specific element, all the elements before it has to be removed.
- ▶ The searching operation also takes a total time of $O(N)$, as reaching any specific element isn't possible without popping the elements stored before it.
- ▶ Operations like insertion or deletion in a stack take constant time i.e. $O(1)$.

Time complexity analysis

Link List:

- ▶ For insertion in the linked list, the time complexity is $O(1)$ if done on the head, $O(N)$ if done at any other location, as we need to reach that location by traversing the linked list.
- ▶ For deletion, the time complexity is $O(1)$, if done on the head, $O(N)$, if done at any other location, as we need to reach that location by traversing the linked list.
- ▶ For searching and accessing any elements, the involved time complexity is $O(N)$.

Data Structure	Insert	Delete	Access	Search
Array	$O(N)$	$O(N)$	$O(1)$	$O(N)$
String	$O(N)$	$O(N)$	$O(1)$	$O(N)$
Queue	$O(1)$	$O(1)$	$O(N)$	$O(N)$
Stack	$O(1)$	$O(1)$	$O(N)$	$O(N)$
Linked List	$O(1)$, if inserted on head $O(N)$, elsewhere	$O(1)$, if deleted on head $O(N)$, elsewhere	$O(N)$	$O(N)$